

FH Münster, Fachbereich Maschinenbau

Hausarbeit im Modul Moderne Konzepte der Programmierung

Entwicklung einer Gaze-Tracking-Anwendung

Von: Oliver Lux und Marek Brackmann
Matrikelnummer: 1423985 ,1476105

Betreuer: Prof. Dr.-Ing. Stephan Behr

Münster, 22. September 2025

Inhaltsverzeichnis

1	Einleitung (Marek Brackmann)	1
2	Eye-Tracking (Marek Brackmann)	3
2.1	Kurzbeschreibung der Anwendung	3
2.2	Programmarchitektur	4
2.3	Algorithmische Grundlagen zur Pupillendetektion	8
2.4	Implementierungsdetails zur Pupillendetektion	10
2.5	Ergebnisse und Bewertung	12
3	Gaze-Tracking (Oliver Lux)	13
3.1	Zielsetzung	13
3.2	Szenarien	13
3.2.1	Normalbetrieb	13
3.2.2	Kalibrierung	13
3.2.3	Mögliche Fehlerfälle	14
3.2.4	Erweiterungen	14
3.3	Programmarchitektur	15
3.4	Klassenbeschreibungen	17
3.4.1	Mainwindow	17
3.4.2	Filter	17
3.4.3	CameraHandler	18
3.4.4	Kalibrierung	18
3.4.5	EyePositionPainter	19
3.5	Entwurfsmuster Beobachter	20
4	Fazit (Oliver Lux)	22
	Literatur	23

1 Einleitung (Marek Brackmann)

Gaze-Tracking ist ein Verfahren, bei dem anhand der Blickrichtung einer Person der Punkt bestimmt wird, auf den sie schaut. In den meisten Fällen soll dabei der Blickpunkt auf einem Bildschirm ermittelt werden. Ein typischer Anwendungsfall des Gaze-Tracking ist die Optimierung von Websites. Dabei wird untersucht, wohin Probanden auf einer Website schauen, um Rückschlüsse über deren Nutzung und Verhalten zu ziehen.



Abbildung 1: Gazetracking Heatmap [1]

In klassischen Umsetzungen kommt dafür meist eine Infrarotkamera zum Einsatz, die in einem Brillengestell so platziert ist, dass sie die Augen des Nutzers erfasst. Daraus kann im Anschluss die Blickrichtung berechnet werden. Diese Methode liefert präzise und zuverlässige Ergebnisse, ist jedoch mit zusätzlichen Kosten verbunden. Eine kostengünstigere Alternative ist die Nutzung der in den meisten Notebooks integrierten Kamera, um die Blickrichtung zu bestimmen.

Ziel der Hausarbeit ist die Entwicklung einer Gaze-Tracking-Anwendung mithilfe dieser Notebook-Webcam. Der Blickpunkt des Nutzers auf dem Bildschirm soll ermittelt und sichtbar gemacht werden. Das Vorgehen umfasst folgende Schritte:

1. Aufnahme eines Nutzerbildes über die Notebook-Webcam
2. Bestimmung der Pupillenmitte per Eye-Tracking
3. Berechnung der Relation zwischen Pupillenposition und Blickpunkt
4. Visualisierung des Blickpunkts auf dem Bildschirm

Um die Aufgabe zu bewältigen, wurde diese in zwei Teile aufgeteilt. Der erste Aufgabenteil besteht darin die Pupillenmitte der beiden Augen durch Bildverarbeitungsmethoden zu bestimmen (Marek Brackmann). Der zweite Teil besteht darin mithilfe der Pupillenmitten den Blickpunkt auf dem Bildschirm zu approximieren und ihn anzuzeigen (Oliver Lux).

Die Umsetzung erfolgt in C++ unter Anwendung von objektorientierter Programmierung. Für die Erstellung einer grafischen Benutzeroberfläche wird der Qt-Creator in Verbindung mit der qt6-Bibliothek genutzt. Zudem wird die OpenCV2-Bibliothek

verwendet, um die Bildverarbeitung zu vereinfachen. Desweiterem werden eine abstrakte Klasse und ein Entwurfsmuster eingesetzt, um den Code übersichtlicher zu gestalten.

2 Eye-Tracking (Marek Brackmann)

Dieses Kapitel behandelt die entwickelte Eyetrackinganwendung auf deren Grundlage die Gazetracking basiert.

2.1 Kurzbeschreibung der Anwendung

Die Anwendung wertet die gelieferten Frames einer Notebookwebcam in Echtzeit aus, erkennt darin Gesicht und Augen und berechnet aus den erkannten Augen die Pupillenmittelpunkte. Dafür greift sie auf die Webcam zu, zerlegt das Video in einzelne Frames und analysiert jeden Frame. Ein Detektor lokalisiert das Gesicht aus dem Originalframe, ein weiterer die Augen aus dem Gesicht und der letzte die Pupillenmitte aus den Augen.

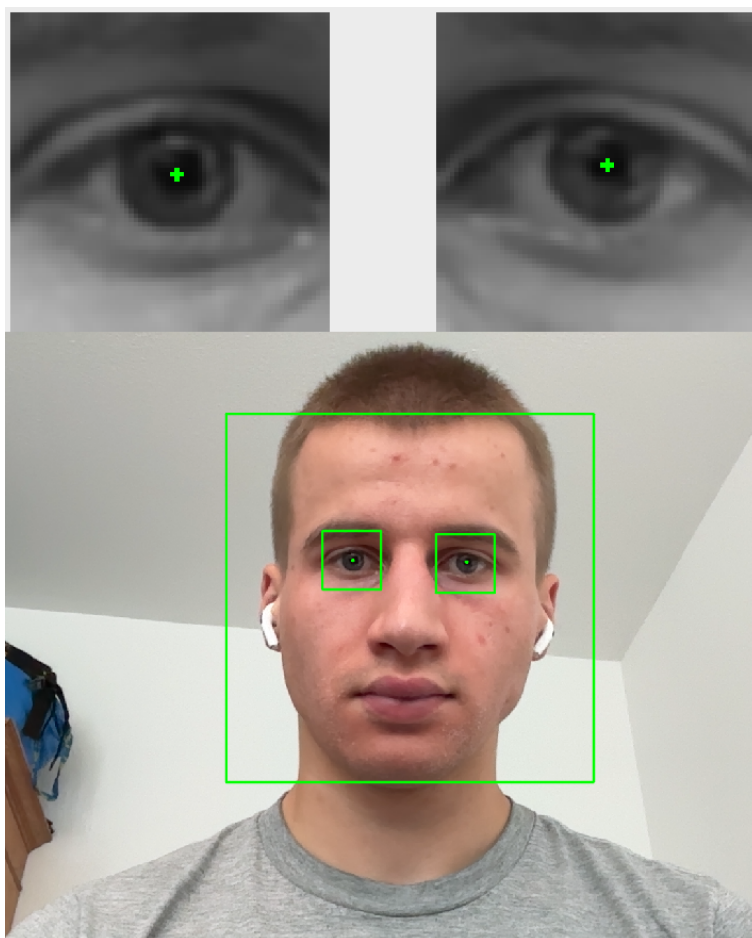


Abbildung 2: Ergebnis der Anwendung

Zur Funktionsüberprüfung werden die Ergebnisse als Rechteck für Gesicht und Augen sowie Punkten für die Pupillenmittelpunkte live auf das Originalbild überlagert. Um die Zentrierung der Mittelpunkte besser zu überprüfen, werden zusätzlich vergrößerte Nahaufnahmen der Augen mit ihren markierten Pupillenmittlen angezeigt.

Die dabei entstehenden Pixelkoordinaten der Pupillenmittelpunkte bilden die Schnittstelle für die Gaze-Tracker-Anwendung (von Oliver Lux), welche daraus den Blickpunkt auf dem Bildschirm approximiert. Die beschriebene Visualisierung ist vor allem für Funktionsprüfungen gedacht und im später für das Gaze-Tracking nicht mehr erforderlich. aefwefwef

2.2 Programmarchitektur

In der Abbildung 3 sieht man ein UML-Diagramm der Eyetracking Anwendung. Es besteht aus neun Klassen:

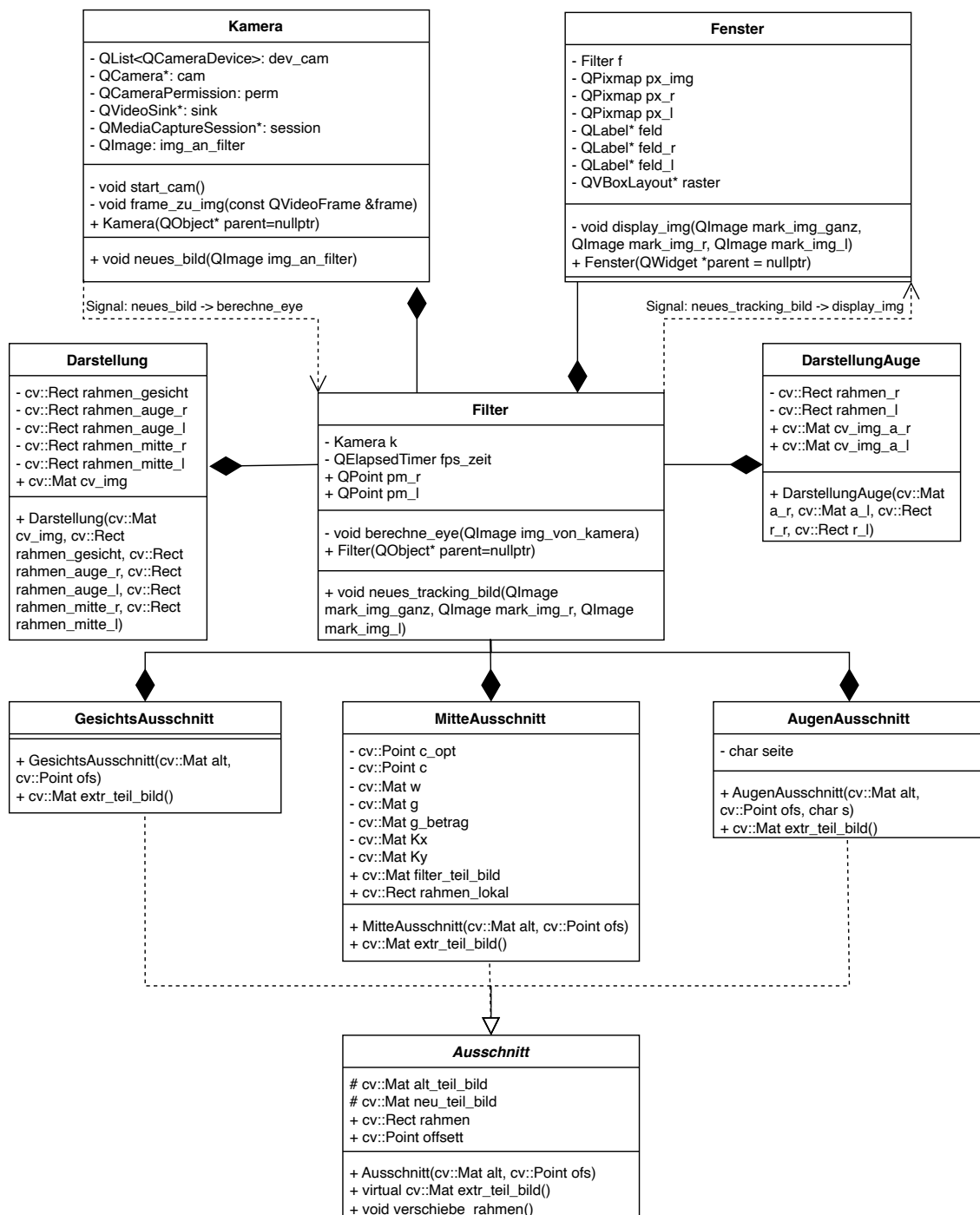


Abbildung 3: Programmarchitektur als UML-Diagramm

1. Kamera: Startet die Kamera und sendet Frames an Filter
2. Filter: Bestimmt die Pixelkoordinaten der Pupillenmittelpunkte und erstellt Ori-

ginalbild mit Markierungen sowie Detailausschnitte der Augen.

3. Fenster: Zeigt das markierte Originalbild sowie Detailausschnitte der Augen an
4. Ausschnitt: Abstrakte Klasse für alle Ausschnitte. Im Konstruktor wird immer eine ROI (region of interest) übergeben und daraus eine konkreter, kleinere ROI bestimmt.
5. Gesichtsausschnitt: Bestimmt Gesichts-ROI aus dem originalen Bild
6. AugenAusschnitt: Bestimmt die Augen-ROI aus der Gesichts-ROI
7. MitteAusschnitt: Bestimmt den Pupillenmittelpunkt-ROI aus den Augen ROI
8. Darstellung: Überlagert das originale Bild mit Markierungen für Gesicht, Augen und Mitten
9. DarstellungAuge: Überlagert das Bild der Augen-ROI mit Markierungen für die Pupillenmitte

Beim Start von main wird das erzeugt das Fenster-Objekt instanziiert und maximiert angezeigt. Das Fenster enthält ein Filter-Objekt “f”, dessen Konstruktor wiederum instanziiert ein Kamera-Objekt “k”. Diese Kamera richtet die Hardware ein. Sie wählt das erste verfügbare Videogerät “dev_cam”, beantragt die macOS- Kameraberechtigung “perm” und startet die Kamera “cam”. Durch eine Qt-connect-Methode in der Kameraklasse wird jedes Videoframe an die Methode “Kamera::frame_zu_img()” gereicht. Dort wird das Frame in ein QImage konvertiert, horizontal gespiegelt, damit sich die Anzeige wie ein Spiegel verhält und als Signal “neues_bild(QImage)” ausgesendet.

Wird durch die Methode “neues_bild(QImage)” ein QImage des Kameraframes gesendet wird die Methode “berechne_ey(QImage)” in der Klasse Filter ausgeführt. Sie bekommt das Bild übergeben und konvertiert es zunächst in ein 1-Kanal (Graustufen) Bild im format cv::Mat. Danach werden in Reinforme Objekte der Klassen Gesichtsausschnitt, AugenAusschnitt, MitteAusschnitt instanziiert. Alle diese Klassen sind abgeleitet von der abstrakten Klasse Ausschnitt und enthalten die abstrakte Methode “extr_teil_bild”. Die Methode “extr_teil_bild()” verwendet die vom vorherigen Objekt berechnete ROI “alt_teil_bild” um eine neue ROI neu_teil_bild zu berechnen. Dabei wird vom vorherigen Objekt berechneten ROI, dem aktuellen Objekt über den Konstruktor übergeben.

Die eigentliche Bildverarbeitung erfolgt dann wie folgt: Zuerst bildet Gesichtsauschnitt aus dem Graubild einen Rechteckrahmen um das Gesicht. Dazu lädt die Klasse eine Haar-Cascade aus der openCV-Bibliothek. Dies Cascade und das Originalbild “alt_teil_bild” wird an die OpenCV-Methode detectMultiScale übergeben. Die Methode detektiert alle Gesichter im Bild und gibt eine Liste von cv::Rect Rechtecken zurück. Jedes Rechteck beschreibt ein Bereich, in dem sich ein Gesicht befindet. Werden mehrere Gesichter erkannt, wählt sie das größte Gesicht aus. Wird kein Gesicht erkannt, setzt sie das Rechteck aus Robustheitsgründen auf das ganze Bild. Der Bereich des ausgewählten Gesichtes wird im Objekt “Rahmen” gespeichert. Aus dem Originalbild wird dann mittels dieses Rahmens das Gesichtsteilbild “neu_teil_bild” ausgeschnitten. Gleichzeitig pflegt die Klasse einen Positionsverschiebungsvariable “offsett”. Jeder Ausschnitt speichert nicht nur den Rahmen mit der Position im KS des aktuellen Teilbilds,

sondern transformiert ihn über die Methode “verschiebe_rahmen()” mit der darin genutzte Variable “offsett” in das Koordinatensystem des ursprünglichen Kamerabildes (Globales Koordinatensystem). So lassen sich hinterher alle Markierungen leicht im Originalbild darstellen.

Als Nächstes werden zwei AugenAusschnitt-Objekte mit dem “neu_teil_bild”, dem “offsett” der Gesichtsausschnitt-Klasse und ein Indikator “seite” instanziiert. Je nach Instanziierung durch Seite ‘r’ oder ‘l’ wählt die Klasse eine dieser Hälften des Gesichts und sucht darin mit einer Augen-Cascade nach dem Auge. Wenn nichts gefunden wird, fällt sie auf die ganze Gesichtshälfte zurück, bei mehreren Treffern nimmt sie das größte Rechteck. Genau wie in der Gesichtsausschnitt-Klasse. Das Ergebnis sind zwei Teilbilder der Augen “neu_teil_bild” und jeweils die globalen Koordinatenrahmen “rahmen” sowie ein neues “offsett” für die nächste Instanziierung der MitteAusschnitt-Objekte.

Die dritte Stufe, MitteAusschnitt, bestimmt die Pupillenmitte im jeweiligen Auge. Hier werden erneut zwei Objekte mit dem “neu_teil_bild”, dem “offsett” der AugenAusschnitt-Klasse instanziiert. Einmal für die Rechte und einmal für die linke Pupillenmitte. Die Pupillenmitte wird in der Klasse nicht wie in den anderen Klassen Gesichtsausschnitt und AugenAusschnitt mit einer Cascade bestimmt, sondern durch einen Ansatz von Timm und Barth [2]. Dieser nutzt die Gradienten der Augenbilder, um den Mittelpunkt der Pupille zu bestimmen. Im Nächsten Kapitel wird dieses Verfahren näher beschrieben. Das Ergebnis der Berechnung sind die Pupillenmittelpunkte. Die werden jeweils im Objekt “Rahmen” gespeichert. Dabei hat diesmal das Rechteck des Rahmens eine Breite und Höhe von einem Pixel. Die Koordinaten des 1x1-Rahmens werden in einem letzten Schritt wieder durch das Offsett in das globale Koordinatensystem transformiert.

Nach dem Berechnen der Rahmen für Gesicht, Augen und Pupillenmittelpunkte werden in der Methode “berechne_eyeball” zwei weitere Objekte der Klassen “Darstellung” und “DarstellungAuge” instanziiert. Das Objekt der Klasse Darstellung erhält im Konstruktor die Rahmen von Gesicht “rahmen_gesicht”, der der beiden Augen “rahmen_auge_r”, “rahmen_auge_l” und der beiden Pupillenmittelpunkten “rahmen_mitte_r” und “rahmen_mitte_l”, sowie das Originalbild der Kamera “cv_img”. Das Objekt von DarstellungAugen erhält im Konstruktor die Rahmen der Pupillenmittelpunkte “rahmen_r”, “rahmen_l” sowie die Teilbilder der Augen “cv_img_a_r”, “cv_img_a_l”. In beiden Objekten werden die Rahmen auf das übergebene Bild überlagert.

Zusätzlich berechnet in der Methode “berechne_eyeball” eine QElapsedTimer- Stoppuhr aus Zeit die benötigt wurde um für einen Frame die Methode “berechne_eyeball(QImage)” auszuführen die aktuelle Bildrate (FPS). Diese wird aus Debugzwecken in der Konsole ausgegeben.

Zuletzt werden die erzeugten Bilder mit Markierungen und die Berechneten Pupillenmittelpunkte von cv::Mat in QImage und cv::Rect in QPoint umgewandelt. Die Pupillenmittelpunkte werden im Format QPoint “pm_r” und “pm_l” gespeichert und stellen die Schnittstelle für nachgelagertes Gaze-Tracking dar. Das durch das Darstellung-Objekt generierte Bild “mark_img_ganz” und die durch das DarstellungAuge-Objekt generierten Bilder “mark_img_r”, “mark_img_l” mit dem Signal “neues_tracking_bild()” an das Fenster gesendet.

Das Fenster enthält drei QLabel eines für das große Kamerabild und zwei für die Augenansichten. In Methode “display_img()” wandelt es die QImage in QPixmap, skaliert sie für die Anzeige und steckt die Pixmaps in die Labels. Die Methode wird immer dann ausgeführt, wenn das Signal “neues_tracking_bild()” gesendet wird. Die Augen-Labels sind über dem Hauptbild positioniert, sodass im laufenden Betrieb gleichzeitig das videobasierte Ergebnis und die Auge-Detailansicht gezeigt wird.

Durch die Darstellung der ROI von Gesicht und Augen durch die eingeblendeten Rahmen, der Detailansicht der beiden Augen mit den berechneten Pupillenmitten und der Ausgabe der FPS durch die Konsole ließ sich der Eye-Tracking-Algorithmus in Bezug auf Zuverlässigkeit und Performance optimieren.

2.3 Algorithmische Grundlagen zur Pupillendetektion

Die Pupille ist kreisförmig und dunkler als die Umgebung. Bestimmt man die Gradienten g des Augenausschnitts erkennt man einen sehr großen Gradienten am Übergang der weißen Sklera auf den dunklen Iris. Alle diese Gradienten zeigen diese an der Irisgrenze radial vom Pupillenzentrum weg. Bestimmt man den Schnittpunkt an dem sich diese Gradientengerade (mit dem Gradienten als Richtungsvektor) schneiden erhält man die Pupillenmitte.

Da die Pupille nicht perfekt rund ist und nur durch wenige Pixel nicht perfekt kreisförmig repräsentiert wird schneiden sich nicht alle Geraden in einem exakten Mittelpunkt. Um dieses Problem zu umgehen wird jeder Pixel im Bild durch eine Funktion gewichtet. Für jede Gerade, die durch ihn durchläuft, erhöht sich seine Gewichtung stark, für eine Gerade, die ihn knapp verfehlt erhöht sich seine Gewichtung leicht, für eine Gerade die mehr als 90° von ihm weg zeigt, erhöht sich seine Gewichtung gar nicht. Der Pixel mit der höchsten Gewichtung ist die Pupillenmitte. Mathematisch lässt sich folgendes Bild skizzieren.

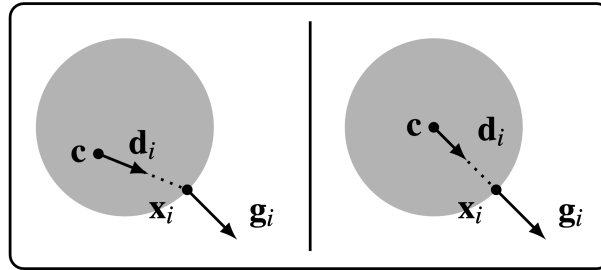


Abbildung 4: Skizze Pupillenmittenbestimmung [2]

Der dunkle Bereich stellt die Iris da der Weiße die Sklera. c ist ein Pixel, der gewichtet wird (also eine mögliche Pupillenmitte). g ist ein Gradient am Punkt x der Irisgrenze. Im linken Teilbild soll dem Pixel c eine geringe Gewichtung addiert werden, da die Gradientengerade den Pixel c verfehlt. Im rechten Teilbild soll eine hohe Gewichtung addiert werden, da die Gradientengerade den Pixel c schneidet. Mathematisch gesehen lässt sich die Gewichtung wie folgt berechnen [2].

$$\mathbf{c}^* = \arg \max_{\mathbf{c}} \left\{ \frac{1}{N} \sum_{i=1}^N (\mathbf{d}_i^T \mathbf{g}_i)^2 \right\} \quad (1)$$

Wobei der Displacementvektor d ein normierter Vektor zwischen gewichtetem Pixel c und dem Stützpunkt/Pixel x ist. Das Skalarprodukt ist Maximal, wenn d und g in dieselbe Richtung zeigen, bei einem Winkel von 90° ist es 0. Negative Skalarprodukte (Winkel größer als 90°) werden ignoriert. Für einen stärkeren Ausschlag wird das Skalarprodukt zusätzlich quadriert. Der maximal gewichtete Pixel c^* (das maximale Argument) aller gewichteten Pixel c ist die Pupillenmitte. Plottet man die Gewichtung über alle Pixel erhält man folgendes Bild.

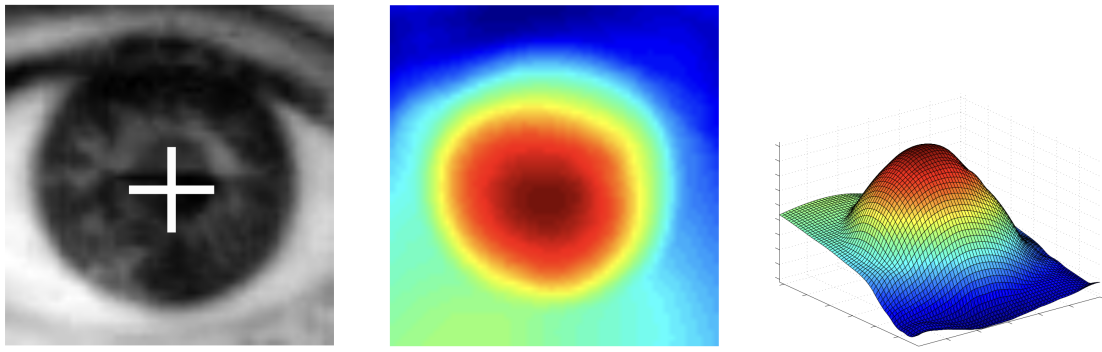


Abbildung 5: Pupille als Funktion [2]

Die Idee besteht also darin das Maximum dieser Funktion zu finden. Wichtig zu erwähnen ist das bei der Berechnung nicht nur Gradienten genutzt werden, die sich an der Irisgrenze befinden, da der Verlauf der Grenze nicht bekannt ist. Daher werden alle Gradienten zur Berechnung der Pupillenmitte c^* genutzt die sich im Augenausschnitt befinden. Da jedoch der die meisten großen Gradientengerade von der Irisgrenze aus die Pupillenmitte schneiden, erhält diese immer noch die größte Gewichtung. Die Gradientenbeträge eines Augenausschnitts (inkl. Markierter Pupillenmitte) sind hier einmal veranschaulicht.

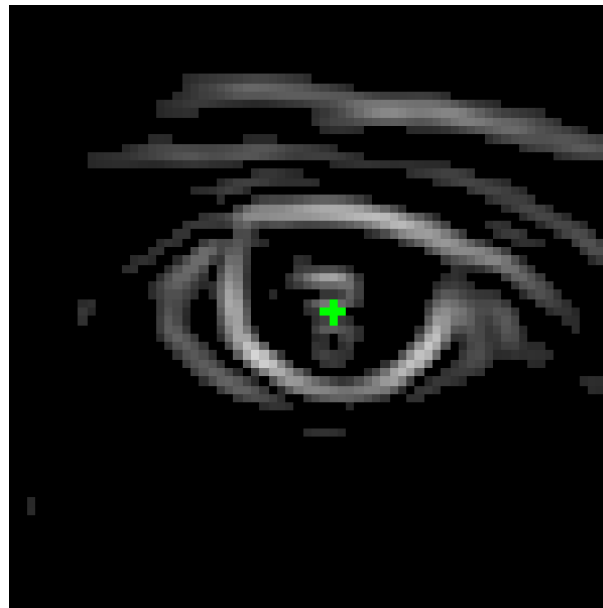


Abbildung 6: Gradienten des Augenausschnitts

Gradienten, die durch die Augenbrauen oder durch Reflektionen im Pupillenzentrum entstehen sind ebenfalls deutlich zu erkennen und fließen mit in die Berechnung ein.

2.4 Implementierungsdetails zur Pupillendetektion

Zu Beginn wird der erkannte Augenausschnitt auf eine Größe von 70×70 Pixel herunterskaliert um den Rechenaufwand, den das Verfahren benötigt deutlich zu reduzieren. Anschließend glättet ein Gaußfilter das Bild, wobei die Kernelgröße k abhängig von der Breite des Augenausschnitts in Pixeln gewählt wird. Danach werden die Gradienten des Bildes mittels Faltung mit den 3×3 -Sobel-Kernen berechnet und der Gradientenbetrag bestimmt. Alle Pixel, deren Gradientenbetrag weniger als 0,3 Standardabweichungen über dem Mittelwert liegt, werden auf null gesetzt [3]. Dadurch tragen nur hinreichend starke Kanten zur späteren Zentrumssuche bei. Im nächsten Schritt wird ein Helligkeitsschwellenwert "w_schwell_5p" festgelegt. Er entspricht dem Wert, über dem die dunkelsten 5% der Pixel im Augenausschnitt liegen. Dieser Schwellwert wird später genutzt, um den Rechenaufwand weiter zu reduzieren. Zur Bestimmung der Pupillenmitte kommt anschließend das Verfahren aus dem vorherigen Kapitel zum Einsatz.

```
for (int cy = 1; cy < zeilen - 1; ++cy) {
    for (int cx = 1; cx < spalten - 1; ++cx) {
        float w_c = w.at<float>(cy, cx);
        if (w_c <= w_schwell_5p){continue;}
        float summe = 0;

        for (int py = 1; py < zeilen - 1; ++py) {
            for (int px = 1; px < spalten - 1; ++px) {
                cv::Vec2f g_p = g.at<cv::Vec2f>(py, px);
                if (px == cx && py == cy){continue;}
                if (g_betrags.at<float>(py, px) == 0)
                    {continue;}

                float d_cp_x = (float)(px - cx);
                float d_cp_y = (float)(py - cy);
                float laenge = std::sqrt(d_cp_x*d_cp_x +
                                         d_cp_y*d_cp_y);

                d_cp_x = d_cp_x/laenge;
                d_cp_y = d_cp_y/laenge;

                float summand = d_cp_x * g_p[0] +
                               d_cp_y * g_p[1];
                if (summand > 0) {
                    summe += summand * summand;
                }
            }
        }

        if (summe > summe_opt) {
            c_opt.x = cx;
            c_opt.y = cy;
            summe_opt = summe;
        }
    }
}
```

Im Inneren der Schleifen wird der Displacementvektor “d_cp” aus den Positionen von Punkt c und p gebildet. Dieser Vektor wird normiert und das Skalarprodukt mit dem Gradienten “g_p” am Punkt p berechnet. Ist das Skalarprodukt “summand” positiv, also der Winkel zwischen den Vektoren kleiner als 90° , wird das resultierende Gewicht dem Gesamtgewicht “summe” des Pixels c hinzugefügt. Übersteigt das so neu berechnete Gewicht “summe” den bisherigen Maximalwert “summe_opt”, wird die Position des Pixels c als neue vorläufige Pupillenmitte gespeichert.

Um den Rechenaufwand zu verringern, werden jene Iterationen übersprungen, bei denen bestimmte Bedingungen nicht erfüllt sind. Dazu zählt, dass ein Pixel c nicht zu den dunkelsten fünf Prozent des Bildes gehört, da die Iris im Gegensatz zu Haut und Sklera sehr dunkel ist und daher als Suchraum eingegrenzt werden kann. Ebenso wird eine Iteration verworfen, wenn der Gradient an der betreffenden Stelle null ist, da das Skalarprodukt in diesem Fall ebenfalls null wäre und somit kein Beitrag zur Gewichtung entsteht. Schließlich werden auch die Fälle ausgelassen, in denen das gewichtete Pixel c direkt mit dem Gradientenpunkt p zusammenfällt, da hier kein Displacementvektor gebildet werden kann.

Wenn kein Auge erkannt wird, ist der Ausschnitt nicht quadratisch, sondern ein beliebiges Rechteck. In diesem Fall wird der Pupillenmittelpunkt auf die Koordinaten (0, 0) gesetzt. Andernfalls startet das beschriebene Verfahren.

2.5 Ergebnisse und Bewertung

Die Funktion des Trackers zeigt, dass Eye-Tracking mit einer Notebookwebcam grundsätzlich praxistauglich ist. Der entwickelte Ansatz zerlegt die Aufgabe in klar getrennte Schritte. Zuerst wird das Gesicht detektiert, anschließend die Augen, und zum Schluss werden aus den Augenbildern die Pupillenmitten berechnet. Technisch basiert die Umsetzung auf einer Kombination aus Haar-Cascaden für Gesicht und Augen sowie dem gradientenbasierten Verfahren nach Timm und Barth zur Bestimmung der Pupillenmitte. Die Position von Gesicht, Auge und Pupillenmitte konnte durch Markierungen im Originalbild in Echtzeit durch Markierungen angezeigt werden.

Um den Rechenaufwand zu begrenzen, wird der jeweilige Augenausschnitt vorab herunterskaliert, und mehrere Kriterien sorgen dafür, dass Iterationen in den rechenintensiven Schleifen frühzeitig verworfen werden (z. B. schwache Gradienten, zu helle Kandidatenpixel, identische c-/p-Positionen). Unter diesen Bedingungen erreicht das System auf einem MacBook Air (M3) etwa 8 fps. Durch das Verwerfen von zu hellen Kandidatenpixeln entsteht aber auch ein Problem. Bei ungleichmäßiger Beleuchtung oder sehr hoher Bildschirmhelligkeit entstehen Reflexionen in der Pupille. Diese Reflexionsbereiche werden in der Implementierung nicht als Pupillenmitte gewertet, da entsprechende Kandidatenpixel in der inneren Schleife übersprungen werden. Ein weiteres Problem ist, dass bei starker Schrägstellung des Kopfes das Gesicht zeitweise nicht erkannt wird, daraus folgt, dass die Augen im gesamten Bild gesucht werden, was die Framerate deutlich reduziert.

Ein nächster Schritt wäre, eine systematische Analyse der Genauigkeit der Pupillenmittelpunkt-Schätzung, idealerweise unter Nutzung eines Datensatzes aus Augenbildern.

3 Gaze-Tracking (Oliver Lux)

Dieses Kapitel beschreibt die grundlegende Architektur, um die im vorherigen Kapitel beschriebenen Entwicklungen darzustellen und zu beherbergen. Zunächst wird die Zielsetzung der Architektur und die

3.1 Zielsetzung

Das Hauptziel der Architektur ist es, die rechenintensive Filter-Klasse aus dem vorherigen Kapitel mit den notwendigen Daten zu versorgen und das Ergebnis zu visualisieren. Das Hauptproblem besteht dabei, dass die Filter-Klasse aufgrund ihrer OpenCV-Integration sehr rechenintensiv ist und somit eine Operation des Programms auf einem Thread nicht möglich ist. Aus diesem Grund wird die Methode des Multi-Threadings in der Architektur angewendet. Ebenfalls ist das Bestreben wünschenswert, ein zentrales Interface für den Datenfluss zwischen den vorhandenen Klassen zu betreiben, da somit sichergestellt werden kann, dass der Informationsfluss für etwaige nachträgliche Änderungen klar ist (Vermeidung von Spaghetti-Code).

3.2 Szenarien

In diesem Unterkapitel werden zunächst die Nutzungsszenarien der EyeTracking Anwendung vorgestellt, für die die Architektur ausgelegt wird.

3.2.1 Normalbetrieb

Im regulären Betrieb startet die Benutzerin oder der Benutzer die Anwendung über das MainWindow. Dieses initialisiert alle wesentlichen Komponenten: den Camerahandler, die Bildverarbeitung (Filter), die Kalibrierung sowie die grafische Darstellung der Augenposition (EyePositionPainter). Wird die Kamera gestartet, erzeugt der Camerahandler kontinuierlich Bilder mit einer Bildrate von etwa 30 Bildern pro Sekunde. Diese Bilder werden parallel an zwei Stellen weitergeleitet: Zum einen an die grafische Oberfläche, wo das Rohbild der Kamera angezeigt wird, und zum anderen an den Filter, der die eigentliche Bildverarbeitung durchführt. Der Filter überführt die Bilddaten in eine geeignete OpenCV-Datenstruktur, erkennt Gesicht, Augen und Pupillen und gibt die ermittelten Koordinaten in Form von Signalen weiter. Diese Koordinaten werden vom Darstellungsmodul (EyePositionPainter) empfangen, dort normiert und in die Koordinaten des Overlays umgerechnet. Auf diese Weise wird die Blickposition der Benutzerin oder des Benutzers durch einen markierten Punkt in Echtzeit visualisiert. Aufgrund der rechenintensiven Operationen bedingt durch die Haarcascaden von OpenCV ist es nötig, dass nur vier Bilder pro Sekunde an die Bildverarbeitung weitergegeben werden, damit die Performance des Programms nicht beeinträchtigt wird. Gleichzeitig wird die Bildverarbeitung auf einem separaten Thread durchgeführt, damit die Darstellung des Ergebnisses auf der GUI nicht beeinträchtigt wird.

3.2.2 Kalibrierung

Ein weiterer zentraler Anwendungsfall ist die Kalibrierung. Sie ist notwendig, um individuelle Unterschiede in der Erfassung der Pupillenposition zu kompensieren und die Bildkoordinaten korrekt auf den Bildschirm abzubilden. Während des Kalibrierungsvorgangs werden der Benutzerin oder dem Benutzer nacheinander Referenzpunkte, ty-

pischerweise an den Bildschirmrändern, präsentiert. Bei jedem Schritt wird die aktuelle Pupillenposition aufgezeichnet und in einem Vektor gespeichert. Nach Abschluss dieses Prozesses stehen vier Referenzpunkte zur Verfügung, die ein Kalibrierrechteck aufspannen. Mithilfe dieser Daten können die vom Filter gelieferten Pupillenkoordinaten über Interpolationsverfahren (z. B. bilineare Interpolation) in Bildschirmkoordinaten überführt werden. Die Kalibrierung stellt somit sicher, dass die dargestellte Blickposition mit der tatsächlichen Blickrichtung der Benutzerin oder des Benutzers übereinstimmt.

3.2.3 Mögliche Fehlerfälle

Neben dem regulären Betrieb und der Kalibrierung sind auch mögliche Fehler- und Ausnahmeszenarien zu berücksichtigen. So kann es vorkommen, dass keine Kamera verfügbar ist, wodurch der Camerahandler keine Bilder liefern kann. Ebenso ist es möglich, dass der Filter kein Gesicht oder keine Augen im Kamerabild erkennt, wodurch keine Pupillenpositionen ausgegeben werden. In solchen Fällen werden entweder keine Blickpunkte dargestellt oder das System signalisiert den Fehlerzustand. Auch eine unvollständige Kalibrierung, beispielsweise durch vorzeitigen Abbruch, führt dazu, dass keine gültigen Referenzdaten vorliegen und somit keine zuverlässige Blickpositionsbestimmung erfolgen kann.

3.2.4 Erweiterungen

Darüber hinaus lässt sich das System um verschiedene Erweiterungen ergänzen. Ein besonders relevanter Anwendungsfall ist die Steuerung von Benutzerschnittstellen über die erfasste Blickrichtung. Anstatt den Blickpunkt lediglich grafisch darzustellen, kann dieser als Eingabemodalität genutzt werden. So ist es beispielsweise denkbar, bestimmte Bereiche des Bildschirms mit Funktionen zu verknüpfen, die durch gezieltes Ansehen aktiviert werden. Dies könnte die Auswahl von Schaltflächen ohne Maus oder Tastatur, das Blättern in Dokumenten oder das Starten von Programmen ermöglichen. Eine solche Interaktion kann insbesondere in Kontexten mit eingeschränkten Eingabemöglichkeiten, etwa in Assistenzsystemen für Menschen mit körperlichen Beeinträchtigungen, erhebliche Vorteile bieten. Darüber hinaus eröffnet die kontinuierliche Erfassung der Blickposition neue Möglichkeiten für adaptive Benutzerschnittstellen. Anwendungen könnten dynamisch Inhalte hervorheben, die im aktuellen Fokus der Benutzerin oder des Benutzers stehen, oder kontextabhängige Hilfen anbieten. Auch in industriellen oder medizinischen Umgebungen ließe sich so die Aufmerksamkeit gezielt unterstützen, indem kritische Bereiche eines Systems automatisch überwacht und hervorgehoben werden, sobald der Blick dorthin fällt. Schließlich besteht die Möglichkeit, die erfassten Pupillenkoordinaten aufzuzeichnen und für spätere Analysen zu verwenden. Dadurch lassen sich beispielsweise Studien zum Nutzerverhalten, zur Usability von Software oder zur kognitiven Belastung durchführen. Die Erweiterungen zeigen, dass das System nicht nur für die reine Visualisierung der Blickrichtung geeignet ist, sondern auch als Basis für innovative Interaktionskonzepte dienen kann.

3.3 Programmarchitektur

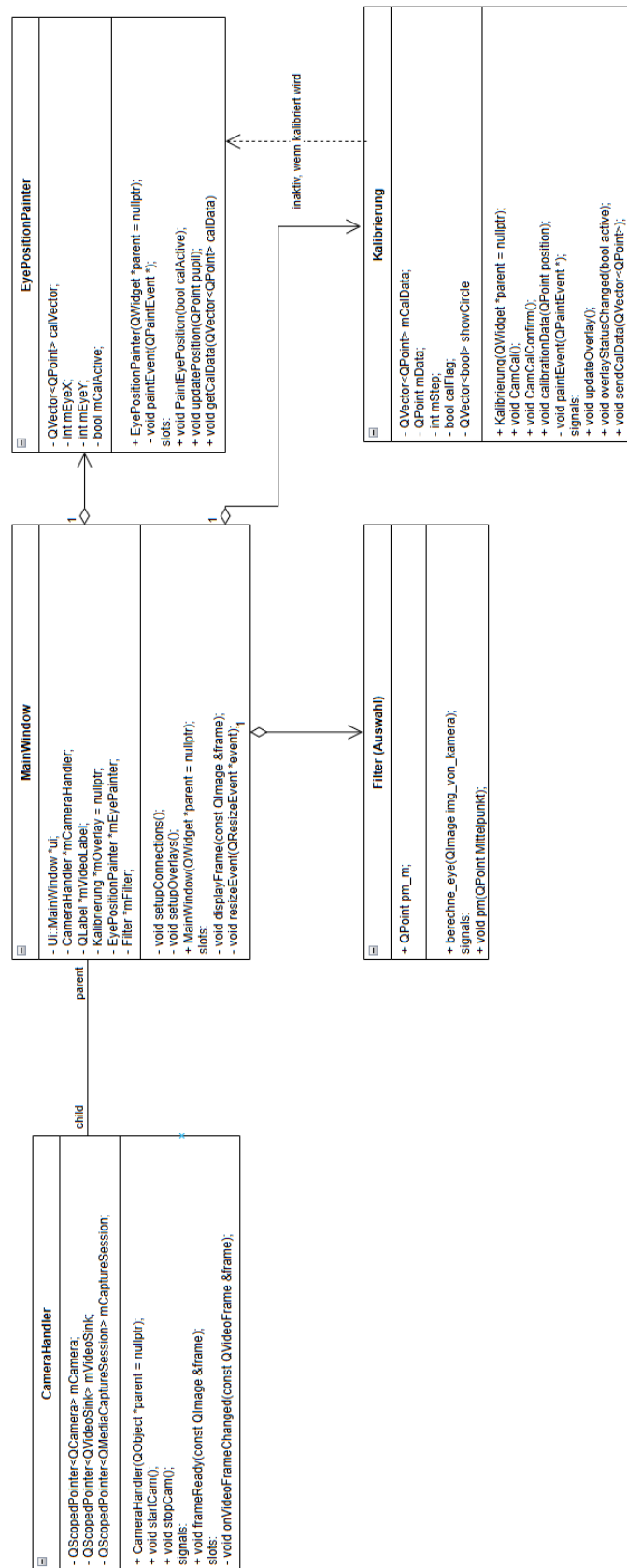


Abbildung 7: Programmarchitektur als UML-Diagramm

In der vorherigen Abbildung ist das UML-Diagramm der Anwendung zu sehen. Zu erkennen ist, dass die MainWindow-Klasse die zentrale Drehscheibe zur Weitergabe von Informationen darstellt. Dies resultiert daraus, dass sämtliche Klassen durch die MainWindow-Klasse instanziiert werden und die Signal/Slot Verbindungen unter den Klassen zentral in der `setupConnections()`-Funktion der MainWindow Klasse aufgesetzt werden.

Die Anbindung an die CameraHandler-Klasse, welche die Konfiguration der Webcam und die Weitergabe des Bildes veranlasst, erfolgt mittels einer Assoziation. Diese Assoziation ist vorhanden, da der Camerahandler das Start- und Stop-Signal aus der MainWindow-Klasse erhält und mit dem `frameReady()`-Signal das Bild zurückliefert. Somit wird die Benutzerschnittstelle (GUI in MainWindow) von der Hardwareinteraktion (Webcam-Zugriff in CameraHandler) sauber getrennt. Diese lose, bidirektionale Kommunikationsbeziehung resultiert in einer engen funktionalen Kopplung ohne Aggregation oder Komposition. In der Implementierung wird hierfür das Signal/Slot-Prinzip genutzt, bei welchem die einzelnen Aktionen in den jeweiligen Klassen Signale aussenden, mit welchen bestimmte Events gestartet werden (z.B. User drückt auf "Kamera starten" → `pbCamStart` wird ausgesendet → `startCam()` in CameraHandler wird aktiviert).

Die Verbindungen zu den Klassen EyePositionPainter, Kalibrierung sowie Filter werden mittels Aggregation realisiert, da innerhalb der MainWindow-Klasse eben jene Objekte erzeugt werden. Die Erzeugung der Objekte wird bedingt durch die Weiterleitung des Webcambilds zur Filter-Klasse, der Pupillenposition zur Kalibrierung und dem EyePositionPainter. Die Weiterleitung der Daten mittels Signal/Slot-Prinzip erzwingt die Erzeugung der Objekte. Dazu ist die EyePositionPainter-Klasse, welche die Pupillenposition aus der Bildverarbeitung auf die Blickposition des Nutzers auf den Bildschirm übersetzt, abhängig davon, ob die Kalibrierung zu einem bestimmten Zeitpunkt aktiv ist. Ist diese aktiv, wird die `calFlag`-Variable auf `true` gesetzt und das Signal an den EyePositionPainter ausgesendet. Dadurch wird die Blickposition des Nutzers nicht dargestellt, während die Kalibrierung aktiv ist.

3.4 Klassenbeschreibungen

3.4.1 Mainwindow

Die Klasse `MainWindow` bildet das zentrale Element der Benutzeroberfläche der Eye-Tracking Anwendung und fungiert als Schnittstelle zwischen dem Nutzer und den zugrunde liegenden Verarbeitungskomponenten. Sie ist für die Darstellung der erfassten Blickposition auf dem Bildschirm verantwortlich und enthält sämtliche Steuerelemente, die für den Betrieb der Anwendung notwendig sind. Dazu gehören beispielsweise Buttons zum Starten und Stoppen der Kamera, zur Initiierung der Kalibrierung sowie weitere Bedienelemente, die eine Interaktion mit der Anwendung ermöglichen.

Neben der rein visuellen Darstellung übernimmt `MainWindow` die Koordination zwischen den einzelnen Softwaremodulen. So hält die Klasse Referenzen auf die Filter-Komponente, die die Pupillenposition berechnet, auf den `CameraHandler`, der den Live-Feed der Webcam bereitstellt, auf die Kalibrierung und den `EyePositionPainter`, welcher die Blickposition auf dem Bildschirm berechnet. Über diese Referenzen werden Befehle, wie das Starten oder Stoppen der Kamera, an die entsprechenden Komponenten weitergegeben, während die zurückgelieferten Daten, beispielsweise die aktualisierten Pupillenkoordinaten, zur Darstellung auf dem Bildschirm genutzt werden.

Die Kommunikation zwischen `MainWindow` und den enthaltenen Komponenten erfolgt über das Signal-Slot-Prinzip von Qt. Benutzeraktionen lösen Signale aus, die von den jeweiligen Modulen empfangen und verarbeitet werden, während die Ergebnisse, etwa neue Bildframes oder kalibrierte Blickpunkte, über Signale an die GUI zurückgemeldet werden. Dadurch wird eine lose Kopplung der Komponenten erreicht, die gleichzeitig eine flexible und modulare Erweiterung des Systems erlaubt. Diese Verbindungen werden in der `setupConnections()`-Funktion zentral aufgerufen.

3.4.2 Filter

Die Filter-Klasse aus dem vorherigen Kapitel wurde dahingehend angepasst, dass nur eine gemittelte Pupillenposition aus dem Augenpaar übergeben wird. Diese gemittelte Pupillenposition wird in der Variable `pm_m` als `QPoint` gespeichert und mit dem Signal `pm(QPoint Mittelpunkt)` übergeben. Diese drei public Operationen stellen das Interface zwischen der Bildverarbeitung, welche bereits im Detail erläutert worden ist, und dem Rest der Anwendung dar. Da diese Operationen sehr rechenintensiv sind, wurde die Berechnung der Filter-Klasse auf einen separaten Thread ausgelagert und nur alle 250 ms durchgeführt. Mithilfe des Signal/Slot-Prinzip konnte dies simpel ausgeführt werden. Die Implementierung der FPS-Limitierung und des Multithreadings ist im Nachgang aufgeführt.

```

QThread* workerThread = new QThread(this);
Filter* worker = new Filter();
worker->moveToThread(workerThread);

connect(mCameraHandler, &CameraHandler::frameReady,
        this, [this, worker](const QImage &frame) {
            static QElapsedTimer timer; //Timer wird hier gestartet
            if (!timer.isValid() || timer.elapsed() > 250) {

                QMetaObject::invokeMethod(worker, "berechne_eye",
                                           Qt::QueuedConnection,
                                           Q_ARG(QImage, frame));

                timer.restart();
            }
        });

workerThread->start();

```

3.4.3 CameraHandler

In der CameraHandler-Klasse wird die Anbindung der Webcam zur Anwendung gesteuert. Diese Klasse wird auf Abruf des "Kamera starten" Pushbuttons der GUI über ein Event aktiviert und über ein separates Event mittels des "Kamera stoppen" Pushbuttons wieder gestoppt. Das Kamerabild wird von einem VideoSink, einer Qt-Klasse, welche die Kamera ansteuert, zu der Ausgangsfunktion `onVideoFrameChanged()` übermittelt. Anschließend werden die Empfänger über das Signal `frameReady()` über ein neues Bild informiert und können dies abrufen.

3.4.4 Kalibrierung

Die Kalibrierung-Klasse beherbergt die Logik für das Overlay, welches die vier Kalibrierkreise in der GUI darstellt. Während das Overlay selbst in der MainWindow-Klasse über dem Kamerabild initialisiert wird, wird jenes von dieser Klasse angesteuert. Als Eingangsgrößen dieser Klasse dienen die beiden Pushbuttons "Kalibrierung starten" und "Bestätigen", sowie die letzte Pupillenposition aus der Filterklasse. Hintergrund der Kalibrierung ist es, dass jeder Nutzer eine eigene Position vor dem eigenen Bildschirm einnimmt und eben jene individuelle Beziehung zwischen Augenposition und Bildschirm standardisiert wird. Dafür soll der Nutzer einmal in jede Ecke des Bildschirms schauen und eben jene Augenposition einmal bestätigen. Ein weiterer Vorteil besteht darin, dass eventuelle Offsets in der Bestimmung der Pupillenposition herausgefiltert werden.

Beim Start des Kalibriervorgangs wird zunächst das Signal ausgesendet, dass die Kalibrierung aktiv ist und die bool-Variable `calFlag` auf wahr gesetzt. Ebenso wird ein 4-Vektor `mCalData` für das Speichern der Blickpositionen in den Ecken initialisiert. Bei jedem Bestätigen wird nun die letzte Pupillenposition abgespeichert und in die nächste Ecke gesprungen. Nach dem viermaligen Bestätigen verschwinden die Kreise in der GUI und `calFlag` wird auf false geschaltet. Somit beginnt wieder die Darstellung der Pupillenposition in Relation zur Kalibrierung, nachdem der 4-Vektor übergeben worden ist.

3.4.5 EyePositionPainter

In dieser Klasse wird nach der Kalibrierung die aktuelle Blickposition des Nutzers auf dem Bildschirm berechnet. Die angewandte Methode hierfür ist die bilineare Interpolation. Als Eingangsgröße dient hier der 4-Vektor mit den Eckdaten aus der Kalibrierung sowie die aktuelle Pupillenposition aus der Filter-Klasse. Die Bildschirmposition wird als QPoint über ein Signal an das dazugehörige Overlay in der MainWindow-Klasse übergeben. Die bilineare Interpolation wurde wie folgt umgesetzt:

```
if (!mCalActive) {

    double relX = double(pupil.x() - calVector[0].x()) /
                  double(calVector[1].x() - calVector[0].x());
    double relY = double(pupil.y() - calVector[0].y()) /
                  double(calVector[2].y() - calVector[0].y());

    relX = std::clamp(relX, 0.0, 1.0);
    relY = std::clamp(relY, 0.0, 1.0);

    // 2) Auf Overlay abbilden
    mEyeX = int(relX * width());
    mEyeY = int(relY * height());

    update(); // Löst Neuzeichnung aus
};
```

Zu sehen ist, dass, wenn die Kalibrierung nicht aktiv ist, die Pupillenposition mit den Randwerten in Relation gesetzt wird. Damit der Kreis, welcher die Blickposition darstellt, im Fehlerfall die Bildschirmfläche nicht verlässt, werden die Relationen in X- und Y-Richtung zwischen 0 und 1 beschränkt. Die Relationen werden dann auf die Bildschirmgröße umgerechnet und dargestellt.

3.5 Entwurfsmuster Beobachter

Das Entwurfsmuster Beobachter beschreibt eine lose Kopplung zwischen Objekten, bei der eine oder mehrere abhängige Instanzen (Beobachter) automatisch über Änderungen des Zustands eines Subjekts informiert werden. In der EyeTracking-Anwendung wird dieses Muster durch die Signal-Slot-Mechanik von Qt umgesetzt. Dabei fungiert die Komponente, die Daten erzeugt, als Subjekt (auch Publisher genannt), während jene, die diese Daten verarbeiten oder visualisieren, die Rolle der Beobachter (Subscriber) übernehmen.

Ein zentrales Beispiel für die Anwendung dieses Musters findet sich in der Interaktion zwischen der CameraHandler-Klasse und dem MainWindow. Der CameraHandler übernimmt die Aufgabe, kontinuierlich neue Bildframes von der Webcam bereitzustellen. Sobald ein neues Bild verfügbar ist, sendet der CameraHandler ein Signal `frameReady()`. Dieses Signal entspricht der Mitteilung des Subjekts im Beobachter-Muster, dass ein relevanter Zustand eingetreten ist – in diesem Fall die Verfügbarkeit eines neuen Kamerabildes.

Das MainWindow registriert sich als Beobachter, indem es das Signal `frameReady()` mit einem passenden Slot verbindet. Dieser Slot ist eine Methode in MainWindow, die die empfangenen Bilddaten entgegennimmt und in der grafischen Benutzeroberfläche anzeigt. Der entscheidende Vorteil dieses Mechanismus liegt darin, dass der CameraHandler nicht explizit wissen muss, welche Klasse seine Daten verarbeitet oder wie viele Beobachter existieren. Es genügt, dass er ein Signal aussendet, das beliebig viele Slots erreichen kann.

Der Ablauf lässt sich anhand eines vereinfachten Beispiels verdeutlichen:

```
// Im CameraHandler:  
emit frameReady(currentFrame);  
  
// Im MainWindow (Registrierung der Beobachtung):  
connect(&cameraHandler, &CameraHandler::frameReady,  
        this, &MainWindow::updateFrame);
```

Hier übernimmt `emit frameReady(currentFrame)` die Rolle des Subjekts, das eine Statusänderung publiziert. Durch den `connect`-Aufruf wird `MainWindow::updateFrame` als Beobachter registriert. Jedes Mal, wenn ein neues Bild vorliegt, ruft das Framework automatisch den entsprechenden Slot auf, ohne dass eine direkte Abhängigkeit zwischen den Klassen besteht.

Durch diese Implementierung wird eine klare Trennung von Verantwortlichkeiten erreicht: Der CameraHandler ist ausschließlich für die Bereitstellung von Bilddaten zuständig, während das MainWindow für die Präsentation zuständig ist. Darüber hinaus eröffnet der Beobachter-Mechanismus die Möglichkeit, weitere Beobachter hinzuzufügen, etwa eine Komponente zur Bildverarbeitung oder Speicherung der Videodaten, ohne den CameraHandler anpassen zu müssen.

Somit trägt die Implementierung des Beobachter-Musters in Form von Signal-Slot-Beziehungen maßgeblich zur Modularität, Flexibilität und Erweiterbarkeit der EyeTracking-Architektur bei. Änderungen oder Erweiterungen in einem Modul haben keine direkten Auswirkungen auf andere, wodurch die Robustheit und Wartbarkeit des Gesamtsystems verbessert werden. Die Gesamtkommunikation der Anwendung mittels `connect` Verbindungen kann der nachfolgenden Abbildung entnommen werden.

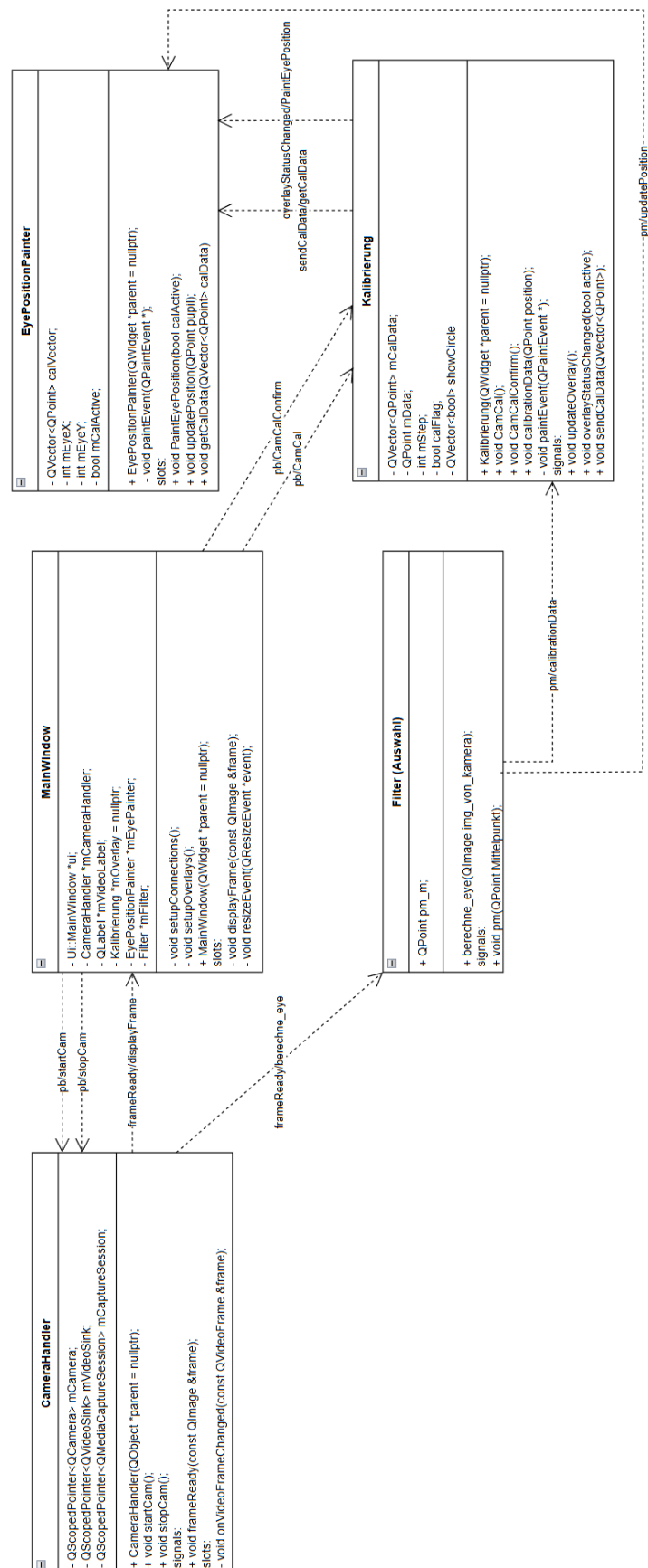


Abbildung 8: Connect-Verbindungen der Anwendung

4 Fazit (Oliver Lux)

Im Rahmen dieses Projekts wurde ein prototypisches System zur Blickpositionsbestimmung auf Basis von Kamerabildern entwickelt. Der zentrale Ansatz bestand darin, Videodaten einer Webcam in Echtzeit zu verarbeiten, Gesicht und Augen zu detektieren und daraus die Pupillenposition zu extrahieren. Durch die Kombination von Qt für die Benutzeroberfläche und Thread-Verwaltung sowie OpenCV für Bildverarbeitung und Objekterkennung konnte eine funktionale Anwendung realisiert werden, die sowohl eine visuelle Ausgabe des Kamerabildes als auch eine Überlagerung mit den berechneten Blickpunkten ermöglicht.

Besonders hervorzuheben ist die klare Trennung der Zuständigkeiten zwischen den Klassen: Der CameraHandler dient als Schnittstelle zur Hardware und stellt kontinuierlich Bilddaten bereit, während der Filter die eigentliche Bildverarbeitung übernimmt. Die Klasse EyePositionPainter visualisiert die Pupillenposition in einem Overlay, wobei die Kalibrierungsklasse zusätzliche Steuerungslogik für eine präzise Zuordnung der Blickrichtung zu Bildschirmkoordinaten bereitstellt. Dieses Zusammenspiel wurde über Signale und Slots realisiert, was eine lose Kopplung und hohe Erweiterbarkeit ermöglicht.

Die Projektumsetzung verdeutlichte auch die Herausforderungen, die mit einer hohen Datenrate von Videoströmen einhergehen. Während der Filter erfolgreich in einen separaten Thread ausgelagert werden konnte, zeigte sich, dass der CameraHandler zunächst im GUI-Thread ausgeführt wurde. Dies führte zu Überlegungen, ob eine weitere Entkopplung in zusätzliche Threads sinnvoll wäre, um die Performance und Stabilität des Gesamtsystems zu erhöhen. Gleichzeitig konnte durch geeignete Mechanismen wie die Reduktion der übergebenen Frames eine Balance zwischen Rechenaufwand und Genauigkeit erzielt werden.

Hinsichtlich der Kalibrierung wurde eine grundlegende Methode implementiert, die vier Referenzpunkte am Bildschirmrand nutzt und mit bilinearer Interpolation eine Zuordnung der Pupillenposition auf den Bildschirm berechnet. Dabei zeigte sich, dass einfache Normalisierungsansätze schnell an ihre Grenzen stoßen, insbesondere wenn das Kalibrierrechteck nicht exakt symmetrisch erfasst wird. Eine zukünftige Erweiterung durch Homographien oder Inverse bilineare Interpolation könnte hier die Genauigkeit erheblich verbessern.

Insgesamt konnte mit diesem Projekt eine funktionierende Grundlage für ein kamerabasiertes Blickpositionssystem geschaffen werden. Der Prototyp ist modular aufgebaut, lässt sich um zusätzliche Bildverarbeitungsverfahren erweitern und kann potenziell in unterschiedlichen Szenarien eingesetzt werden – etwa zur Steuerung von Benutzerschnittstellen, für wissenschaftliche Untersuchungen zur Blickbewegung oder als Assistenzsystem in Mensch-Maschine-Interaktionen. Trotz bestehender Limitierungen in Genauigkeit und Stabilität bildet die Arbeit damit einen soliden Ausgangspunkt für weiterführende Forschung und praxisorientierte Anwendungen.

Literatur

- [1] Victoria Hyzy. *Eye Tracking im digitalen Marketing*. Blogbeitrag auf WebSpotting in der Kategorie „Herausforderungen & Optimierungsmöglichkeiten, Marketing“. 30. Mai 2019. URL: <https://www.webspotting.de/herausforderungen-optimierungsmoeglichkeiten/eye-tracking-im-digitalen-marketing/> (besucht am 06.09.2025).
- [2] Fabian Timm und Erhardt Barth. „Accurate Eye Centre Localisation by Means of Gradients.“ In: Jan. 2011, S. 125–130.
- [3] Tristan Hume. *Simple, accurate eye center tracking in OpenCV*. 2012. URL: <https://thume.ca/projects/2012/11/04/simple-accurate-eye-center-tracking-in-opencv/#fn:2> (besucht am 06.09.2025).